TECHNICAL REPORT v1.3.0

分布式短链接服务设计与实现技术报告

High-Concurrency Distributed ShortLink Service

服务端架构 · 高并发 · 缓存 · 分布式 ID

文档编号	TR-2026-ARCH-014	文档版本	v1.3.0
项目代号	Project Snap	密级	内部公开（Internal）
编写团队	基础架构组 / Platform Engineering	技术栈	Go · Redis · MySQL · Kafka
主要作者	架构师 · 张三	评审人	技术委员会
创建日期	2026-05-20	更新日期	2026-06-11

本文档由 Platform Engineering 团队编制 · 仅限内部技术评审与归档使用

文档信息与修订历史

本章记录文档的版本演进、变更摘要与审批状态，便于追溯设计决策的历史脉络。

版本修订记录

表：文档版本修订历史

版本	日期	修订人	变更摘要	状态
v0.1.0	2026-05-20	张三	初稿，确立整体架构与模块划分	草稿
v1.0.0	2026-05-28	张三 / 李四	补充缓存与分布式 ID 方案，通过初评	已评审
v1.2.0	2026-06-05	王五	新增性能压测数据与优化章节	已评审
v1.3.0	2026-06-11	张三	完善部署运维、监控告警与附录	已发布

术语与缩写

术语 / 缩写	全称	说明
`QPS`	Queries Per Second	每秒查询数，衡量系统吞吐能力
`P99`	99th Percentile Latency	99 分位响应延迟
`TTL`	Time To Live	缓存键的存活时间
`CDC`	Change Data Capture	数据库变更捕获，用于缓存同步
`301/302`	HTTP Redirect	永久 / 临时重定向状态码

> **阅读对象**
>
> 本文档面向后端研发工程师、架构评审委员会及运维（SRE）团队。读者需具备 HTTP、关系型数据库与分布式系统的基础知识。

目录 Contents

1 项目概述与背景 04

- 1.1 问题背景与业务痛点 04
- 1.2 设计目标与非目标 04

2 系统总体架构 05

- 2.3 分层架构设计 05
- 2.4 技术选型与权衡 05

3 核心模块设计与实现 06

3.5 短码生成与分布式 ID 06
3.6 缓存读写与一致性 07
4 接口（API）设计 08
4.7 RESTful 接口规范 08
5 数据库设计 09
6 测试与性能验证 10
7 部署、运维与监控 11
8 总结与展望 12
9 附录 12

1 项目概述与背景

本报告描述了一个面向高并发场景的**分布式短链接服务（ShortLink Service）**的完整设计与实现。系统将长 URL 转换为定长短码，并在用户访问短码时高效地重定向至原始地址，同时具备访问统计、过期失效、防刷与高可用能力。

1.1 问题背景与业务痛点

随着营销短信、社交分享、二维码投放等业务的增长，原始长链接存在以下问题：

- **长度过长**：超出短信字符限制，影响投放成本与可读性；
- **不可统计**：无法精确追踪渠道点击、转化等运营指标；
- **不可控**：链接一经发出无法失效或更换目标地址。

50k+	<10ms	99.99%	10亿+
峰值 QPS（读）	P99 重定向延迟	服务可用性 SLA	短码容量规划

1.2 设计目标与非目标

1.2.1 设计目标

1. **高性能**：重定向路径以缓存为主，P99 延迟控制在 10ms 内；
2. **高可用**：无单点，任一节点故障不影响整体服务；
3. **可扩展**：短码空间与存储均支持水平扩容；
4. **可观测**：完善的指标、日志与链路追踪。

1.2.2 非目标

本期**不**实现链接内容安全审核、自定义域名解析与可视化运营后台，相关能力将在后续迭代中规划。

> **设计原则**
>
> 遵循「读多写少」的访问特征做针对性优化：写入路径保证强一致与唯一性，读取路径以多级缓存换取极致延迟。

2 系统总体架构

2.1 分层架构设计

系统采用经典分层架构，自上而下分为接入层、服务层、缓存层与存储层，各层之间通过明确的接口契约解耦。

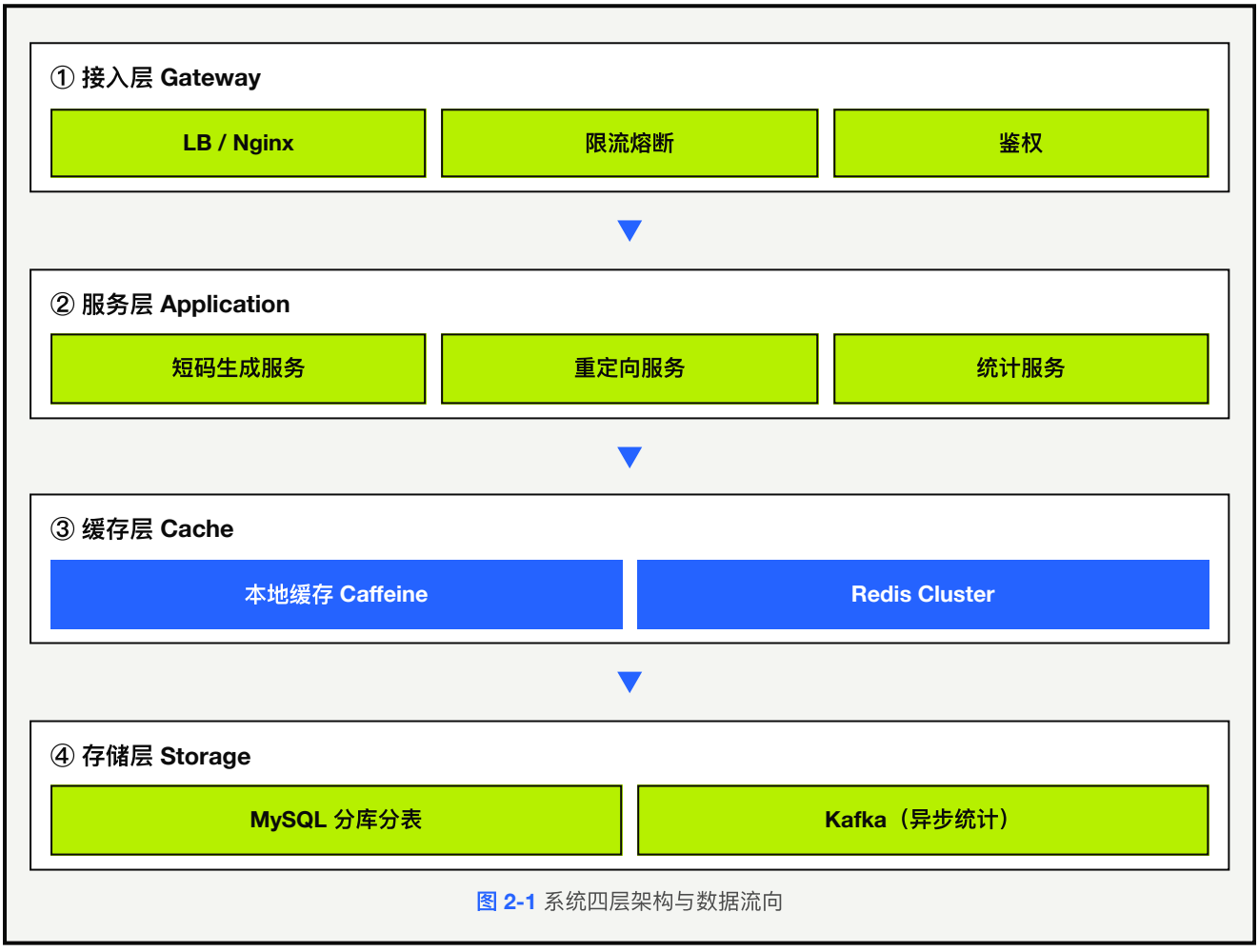


图 2-1 系统四层架构与数据流向

2.2 技术选型与权衡

表：关键技术选型对比

维度	候选方案	最终选择	理由
开发语言	Go / Java	**Go**	高并发协程模型、低内存占用、部署体积小
缓存	Redis / Memcached	**Redis Cluster**	支持持久化、丰富数据结构与原子操作
分布式 ID	UUID / Snowflake / 号段	**号段模式**	有序、可读、对短码进制转换友好
消息队列	Kafka / RabbitMQ	**Kafka**	高吞吐、适合统计类日志的异步落库

> **权衡说明**
> 选择号段模式而非 Snowflake，是为了获得严格递增的数字 ID，从而通过 Base62 进制转换得到更短且无歧义的短码；代价是需要依赖中心化的号段分发服务（已做主备高可用）。

3 核心模块设计与实现

3.1 短码生成与分布式 ID

3.1.1 号段分发原理

ID 分发服务一次性从数据库领取一个号段（如 `[10000, 20000)`），在内存中自增分配，号段耗尽前异步预取下一段，避免请求阻塞。

3.1.2 Base62 短码编码

将自增的十进制 ID 转换为 62 进制（`0-9a-zA-Z`），得到紧凑且大小写敏感的短码。10 亿量级 ID 仅需 6 位字符即可表示。

shortcode.go GO

```go
// 字符表：0-9 a-z A-Z 共 62 个字符
const alphabet = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

// Encode 将自增 ID 编码为 Base62 短码
func Encode(id uint64) string {
    if id == 0 {
        return "0"
    }
    var buf [11]byte
    i := len(buf)
    for id > 0 {
        i--
        buf[i] = alphabet[id%62]   // 取余得到当前位字符
        id /= 62                   // 整除进入下一位
    }
    return string(buf[i:])
}
```

3.2 缓存读写与一致性

3.2.1 多级缓存读取流程

重定向请求依次经过：本地缓存（Caffeine）→ 分布式缓存（Redis）→ 数据库（MySQL）。命中即返回并逐级回填，未命中数据库时写入空值（防穿透）。

resolver.go GO

```go
func (r *Resolver) Resolve(ctx context.Context, code string) (string, error) {
    // L1: 进程内本地缓存
    if v, ok := r.local.Get(code); ok {
        return v.(string), nil
    }
    // L2: Redis 分布式缓存
    url, err := r.redis.Get(ctx, "sl:"+code).Result()
    if err == nil {
        r.local.Set(code, url)        // 回填 L1
        return url, nil
    }
    // L3: 回源数据库 + 防穿透空值
    url, err = r.db.QueryURL(ctx, code)
    if err == sql.ErrNoRows {
        r.redis.Set(ctx, "sl:"+code, "", 60*time.Second)
        return "", ErrNotFound
    }
    r.redis.Set(ctx, "sl:"+code, url, 24*time.Hour)
    return url, err
}
```

3.2.2 缓存一致性策略

场景	策略	说明
新建短链	写库后异步预热	创建成功即写入 Redis，TTL 24h
更新目标地址	Cache Aside + 删除	先更库，再删除缓存键，下次访问回源
缓存击穿	singleflight	同一 key 并发回源仅执行一次
缓存雪崩	TTL 随机抖动	过期时间叠加 ±10% 随机值

> **易错点**
> 更新地址时务必**先更新数据库再删除缓存**，且删除失败需进入重试队列；切勿采用「先删缓存再更库」，否则高并发下可能读到旧值并回填，造成长期脏数据。

4 接口（API）设计

4.1 RESTful 接口规范

所有接口遵循 RESTful 风格，统一返回 JSON，错误码遵循 `{code,message,data}` 包装结构。

表：核心接口一览

方法	路径	说明	鉴权
POST	/api/v1/links	创建短链	需要
GET	/{code}	短码重定向（302）	否
GET	/api/v1/links/{code}	查询短链详情与统计	需要
PUT	/api/v1/links/{code}	更新目标地址 / 过期时间	需要
DELETE	/api/v1/links/{code}	失效短链	需要

4.1.1 创建短链 · 请求与响应示例

POST /api/v1/links JSON

```
// Request Body
{
  "long_url": "https://example.com/products/2026/summer-sale?ref=sms",
  "expire_at": "2026-12-31T23:59:59Z",
  "channel":  "sms-campaign-0611"
}

// Response 200 OK
{
  "code": 0,
  "message": "ok",
  "data": {
    "short_code": "4kZ9aB",
    "short_url":  "https://s.ln/4kZ9aB",
    "created_at": "2026-06-11T11:05:00Z"
  }
}
```

4.1.2 统一错误码

code	HTTP	含义
0	200	成功
40001	400	long_url 非法或缺失
40401	404	短码不存在或已失效
42901	429	触发限流
50001	500	内部服务错误

5 数据库设计

核心表 `short_links` 按 `id` 进行分库分表（16 库 × 64 表），短码经反向映射定位分片。

5.1 表结构定义

schema.sql SQL

```sql
CREATE TABLE short_links (
  id          BIGINT UNSIGNED NOT NULL -- 全局自增ID,
  short_code  VARCHAR(12)      NOT NULL -- Base62短码,
  long_url    VARCHAR(2048)    NOT NULL,
  channel     VARCHAR(64)      DEFAULT NULL,
  status      TINYINT          NOT NULL DEFAULT 1 -- 1正常 0失效,
  expire_at   DATETIME         DEFAULT NULL,
  created_at  DATETIME         NOT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uk_code (short_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2 字段说明与索引策略

字段	类型	索引	说明
id	BIGINT	主键	号段分发，物理有序
short_code	VARCHAR(12)	唯一索引	对外短码，查询入口
status	TINYINT	—	软删除标记
expire_at	DATETIME	普通索引	定时任务清理过期数据

最佳实践

短码查询全部走 `uk_code` 唯一索引，禁止 `long_url` 模糊匹配；过期清理使用 `expire_at` 索引分批删除（每批 1000 行），避免大事务锁表。

6 测试与性能验证

6.1 测试范围

测试覆盖单元测试、接口集成测试与压力测试三个层级，CI 流水线要求单测覆盖率不低于 80%。

6.2 压测环境与结果

表：重定向接口压测结果（4C8G × 3 节点）

并发	QPS	平均延迟	P99 延迟	错误率
1,000	48,200	2.1 ms	6.8 ms	0%
3,000	52,600	4.7 ms	9.2 ms	0%
5,000	53,100	8.9 ms	18.4 ms	0.02%

6.3 性能优化手段

1. **本地缓存**：热点短码命中 L1，免去网络往返；
2. **连接池复用**：Redis / MySQL 连接池预热，避免冷启动抖动；
3. **异步统计**：访问日志投递 Kafka，重定向主链路零落库；
4. **GOMAXPROCS 调优**：容器环境绑定真实 CPU 配额。

> **基准对比**
>
> 引入本地缓存后，P99 延迟由 22ms 降至 9ms，下降约 **59%**；Redis 网络调用量减少约 70%。

7 部署、运维与监控

7.1 容器化部署

Dockerfile DOCKERFILE

```dockerfile
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 go build -o shortlink ./cmd/server

FROM alpine:3.20
COPY --from=builder /app/shortlink /usr/local/bin/
EXPOSE 8080
ENTRYPOINT ["shortlink"]
```

7.2 监控与告警指标

指标	采集方式	告警阈值
重定向 QPS	Prometheus	突降 >50%
P99 延迟	Prometheus	>30ms 持续 1min
缓存命中率	Redis exporter	<90%
5xx 错误率	Nginx 日志	>0.1%

> **灰度发布**
>
> 采用金丝雀策略：先放量 5% 流量观察核心指标 10 分钟，无异常后逐步放量至 100%；任一指标越界自动回滚。

8 总结与展望

8.1 工作总结

本项目实现了一套支持 5 万级 QPS、P99 延迟低于 10ms 的高可用短链接服务。通过号段 ID、Base62 编码与多级缓存的组合，在保证唯一性与一致性的前提下达成了极致的读取性能。

8.2 后续规划

1. **自定义短码**：支持品牌方指定可读短码；
2. **安全审核**：接入 URL 黑名单与内容风控；
3. **多活容灾**：跨可用区单元化部署。

附录

附录 A · 参考资料

1. 《Designing Data-Intensive Applications》, Martin Kleppmann.
2. Redis 官方文档 — Cluster Specification.
3. RFC 7231 — HTTP/1.1 Semantics and Content.

附录 B · 本模板使用说明

如何复用本模板

将各章节替换为你自己的内容即可：标题层级（h1.sec / h2 / h3）会自动编号；代码块用 `.code` 容器并以 `.tk/.ts/.tc/.tf/.tn/.tp` 标注高亮；提示框用 `.box.note/.warn/.danger/.ok`；表格用 `.data`。Word 版结构与此一一对应，可直接在 Word 中编辑。